

ARIN5204: Reinforcement Learning

1.2 Multi-Armed Bandits

Dr. Long Chen

longchen@ust.hk

Assistant Professor, Dept. of CSE

The Hong Kong University of Science and Technology (HKUST)

February 23, 2026



- 1 Multi-Armed Bandits
- 2 Basic Concepts in MAB
- 3 Action Selection in MAB
- 4 Upper Confidence Bounds
- 5 Gradient Bandits

Some slides are from: Katerina Fragkiadaki (CMU), Hado van Hasselt (DeepMind).

① Multi-Armed Bandits

② Basic Concepts in MAB

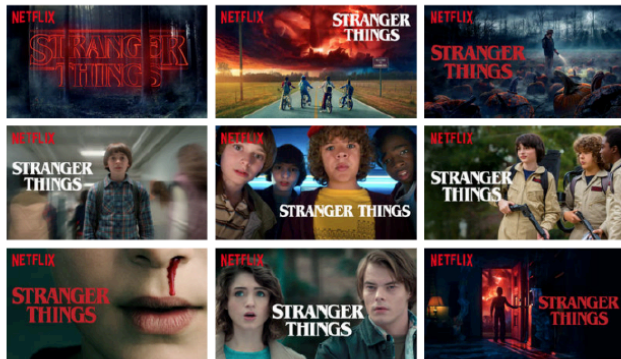
③ Action Selection in MAB

④ Upper Confidence Bounds

⑤ Gradient Bandits

A/B Test

We have multiple choices, and we want to decide which one to display to engage more users, e.g., Netflix users.



This Lecture - Motivation

Learning to act in non-sequential (single action/state) setups

- Each action results in **an immediate reward**
- We want to choose actions that maximize immediate reward in expectation

For example:

- Choosing which restaurant to go to
- **Action**: the restaurants to choose from
- **Rewards**: your pleasure.
- **Q**: What if you become friends with the restaurant employees and enjoy interacting with them?

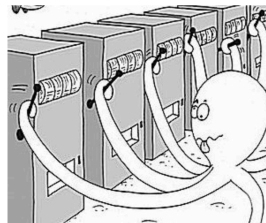
This is not a **stateless** setup! If the state changes as a result of your actions, then we have a sequential setup, and we will address it in the next lecture.

Multi-Armed Bandits

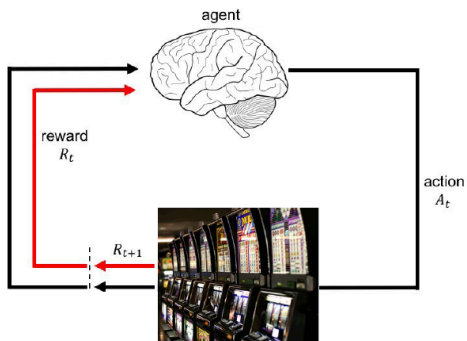
One-armed bandit = Slot machine
(English slang)



Multi-armed bandits = Multiple slot machine



Multi-Armed Bandits



$$a_t, r_{t+1}, a_{t+1}, r_{t+2}, \dots$$

The state does not change! (*a.k.a.*, stateless)

Multi-Armed Bandit Problem

- At each timestep t , the agent chooses one of the K **arms** and plays it
- The k -th arm produces **reward** $r_{k,t}$ when played at timestep t
- The reward $r_{k,t}$ are drawn from a probability distribution $p_k(\cdot)$ with mean μ_k
- The agent does not know neither **reward distributions** nor **their means**
- **Agents Objective:** Maximize cumulative rewards
- We can maximize cumulative rewards over a finite or infinite horizon if I just play the arm with the highest mean reward μ_k each time. (but I don't know which one it is)
- **Definition:** The action-value function for action a is its mean reward:

$$Q_*(a) = \mathbb{E}[r_t | a_t = a]$$

Background: Bernoulli & Gaussian Distribution

- **Bernoulli distribution**

$$\text{Bern}(x|\mu) = \mu^x \cdot (1 - \mu)^{1-x}$$

i.e., $p(x = 1|\mu) = \mu$, $p(x = 0|\mu) = 1 - \mu$.

- The **mean** and **variance**

$$\mathbb{E}[x] = \mu, \quad \text{var}[x] = \mu(1 - \mu)$$

- **Gaussian distribution**

$$\text{Norm}(x|\mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp - \frac{(x - \mu)^2}{2\sigma^2}$$

Example: Bernoulli Bandits

- Each action results in success or failure, and rewards are binary
- Mean reward for each arm represents the probability of success
- Action $k \in 1, \dots, K$ produces a success with probability $\mu_k \in [0, 1]$



μ_1

win 0.6 of time



μ_2

win 0.4 of time

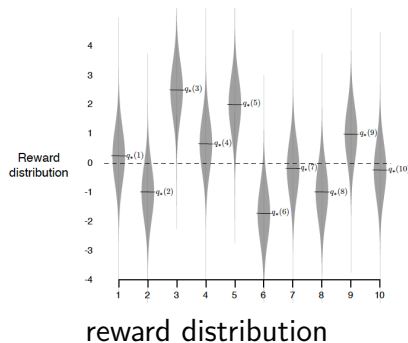


μ_3

win 0.45 of time

Example: Gaussian Bandits

- Example action results in a **real number**
- Action $k \in \{1, \dots, K\}$ produces an average reward equal to the mean of its Gaussian distribution.



- 1 Multi-Armed Bandits
- 2 Basic Concepts in MAB**
- 3 Action Selection in MAB
- 4 Upper Confidence Bounds
- 5 Gradient Bandits

Forming Action-Value Estimates

- We need to estimate the action-value function:

$$Q_t(a) \approx Q_*(a), \forall a$$

- To simplify notation, let us focus on one action:
 - Its action value estimate after $t - 1$ rewards

$$Q_t := \frac{r_1 + r_2 + \dots + r_{t-1}}{t - 1}$$

- Q: the weakness?

Forming Action-Value Estimates (Incrementally)

- How can we do this **incrementally** (without storing all rewards)?
- Could store a running sum, or equivalently:

$$Q_{t+1} = Q_t + \frac{1}{t}[r_t - Q_t]$$

- This is a standard form for learning/update rules

NewEstimation $\leftarrow$ OldEstimate + StepSize [Target – OldEstimate]

Error: Target – OldEstimate

Derivation of Incremental Update

$$Q_t \doteq \frac{r_1 + r_2 + \dots + r_{t-1}}{t-1}$$

$$\begin{aligned} Q_{t+1} &= \frac{1}{t} \sum_{i=1}^t r_i \\ &= \frac{1}{t} \left(r_t + \sum_{i=1}^{t-1} r_i \right) \\ &= \frac{1}{t} \left(r_t + (t-1) \frac{1}{t-1} \sum_{i=1}^{t-1} r_i \right) \\ &= \frac{1}{t} (r_t + (t-1)Q_t) \\ &= Q_t + \frac{1}{t} [r_t - Q_t] \end{aligned}$$

Exploration/Exploitation Dilemma

- We need to estimate the action-value function:

$$Q_t(a) \approx Q_*(a), \forall a$$

- Define the **greedy** action at time t as:

$$a_t^* \doteq \arg\max_a Q_t(a)$$

- If $a_t = a_t^*$, then you are **exploiting**
- If $a_t \neq a_t^*$, then you are **exploring**
- **You can't do both, but you need to do both**
- You can never stop exploring, but maybe you should explore less over time

Exploration/Exploitation Dilemma

- Online decision-making involves a fundamental choice:
 - **Exploitation**: Make the best decision given current information
 - **Exploration**: Gather more information
- The best long-term strategy may involve short-term sacrifices
- Gather enough information to make the best overall decisions
- **The exploration/exploitation dilemma** is not a problem encountered in computational RL or deep RL: *It is a fundamental problem in the decision making of any intelligent agent.*

Evaluation Metric: Regret

- The **action-value** is the **mean reward** for action a ,

$$Q_*(a) \doteq \mathbb{E}[r_t | a_t = a], \quad \forall a \in \{1, \dots, K\}$$

- The **optimal value** is:

$$V_* = \max_a Q_*(a) = Q_*(a_*)$$

- The **regret** is the expected opportunity loss for one step. For an algorithm that selects action a_t at timestep t it reads:

$$l_t = \mathbb{E}[V_* - Q_*(a_t)]$$

Evaluation Metric: Regret

- The **regret** is the expected opportunity loss for one step. For an algorithm that selects action a_t at timestep t it reads:

$$l_t = \mathbb{E}[V_* - Q_*(a_t)]$$

- Gap** Δa is the difference in value between action a and optimal action a_* :

$$\Delta a = V_* - Q_*(a)$$

- The **total regret** is the total opportunity loss:

$$L_T = \mathbb{E} \left[\sum_{t=1}^T V_* - Q_*(a_t) \right]$$

- Maximize cumulative expected reward = minimize total regret

- 1 Multi-Armed Bandits
- 2 Basic Concepts in MAB
- 3 Action Selection in MAB**
- 4 Upper Confidence Bounds
- 5 Gradient Bandits

Fixed Exploration Period + Greedy

- ① Allocate a fixed time period to exploration when you try bandits uniformly at random
- ② Estimate mean rewards for all actions: $Q_t(a) = \frac{1}{N_t(a)} \sum_{i=1}^{t-1} r_i \mathbb{1}(a_i = a)$
- ③ Select the action that is optimal for the estimated mean rewards, breaking ties at random: $a_t = \operatorname{argmax}_a Q_t(a)$

Fixed Exploration Period + Greedy

- After the fixed exploration period, we have formed the following reward estimates



$$Q_t(a_1) = 0.3$$



$$Q_t(a_2) = 0.5$$



$$Q_t(a_3) = 0.1$$

- Q1: Will the greedy method always pick the second action?
- Q2: Can greedy lock onto a suboptimal action forever?
- A2: Greedy has linear total regret

Fixed Exploration Period + Greedy

- 1 Allocate a fixed time period to exploration when you try bandits uniformly at random
- 2 Estimate mean rewards for all actions: $Q_t(a) = \frac{1}{N_t(a)} \sum_{i=1}^{t-1} r_i \mathbb{1}(a_i = a)$
- 3 Select the action that is optimal for the estimated mean rewards, breaking ties at random: $a_t = \operatorname{argmax}_a Q_t(a)$
- 4 GOTO 2 (keep exploration)

ϵ -Greedy Action Selection

- In **greedy** action selection, you always exploit
- In ϵ -greedy, you can usually be greedy, but with probability ϵ you instead pick an action at random (possibly the greedy action again)
- This is perhaps the simplest way to balance exploration and exploitation

ϵ -Greedy Action Selection

A simple bandit algorithm

Initialize, for $a = 1$ to k :

$$Q(a) \leftarrow 0$$

$$N(a) \leftarrow 0$$

Repeat forever:

$$A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{with probability } 1 - \epsilon \\ \text{a random action} & \text{with probability } \epsilon \end{cases} \quad (\text{breaking ties randomly})$$

$$R \leftarrow \text{bandit}(A)$$

$$N(A) \leftarrow N(A) + 1$$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$$

- Q: In the ϵ -greedy action selection, for the case of two actions and $\epsilon = 0.5$, what is the probability that the greedy action is selected.

ϵ -Greedy Algorithm

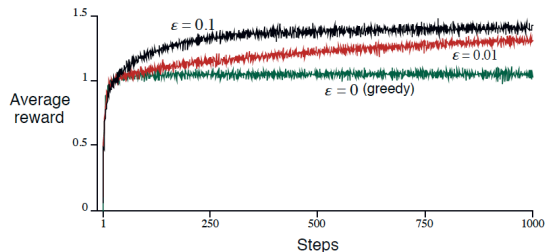
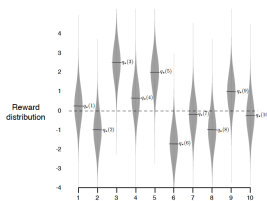
- The ϵ -greedy algorithm continues to explore forever
 - With probability $1 - \epsilon$ select $a_t = \arg \max_a Q_t(a)$
 - With probability ϵ select a random action (independent of its Q estimation)
- Constant ϵ results in the per time step regret to be:

$$I_t \geq \frac{\epsilon}{|\mathbf{A}|} \sum_{a \in \mathbf{A}} \Delta a$$

- ϵ -greedy has linear total regret

Average Reward of Gaussian Bandits

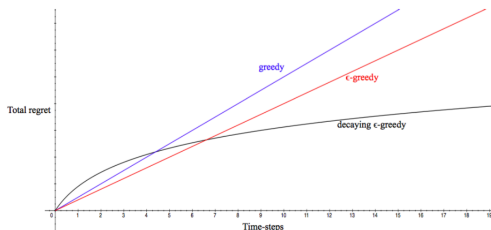
- We sample 2000 times 10-armed bandits: $Q_*(a) \sim N(0, 1)$, $r_t \sim N(Q_*(a), 1)$



- Q1: After infinite steps, which one will result in the largest average reward?
- Q2: Does the performance of those methods depend on the initialization of the action value estimate? (such as optimistic initialization)

Decaying ϵ -Greedy Algorithm

- If an algorithm forever explores, it will have linear total regret
- If an algorithm never explores, it will have linear total regret
- Decaying ϵ -Greedy: design a decay schedule for $\epsilon_1, \epsilon_2, \dots$



We need to reason about **the uncertainty** of our action estimate

- 1 Multi-Armed Bandits
- 2 Basic Concepts in MAB
- 3 Action Selection in MAB
- 4 Upper Confidence Bounds**
- 5 Gradient Bandits

Uncertainty

Uncertainty: The problem with using mean estimates is that we do not reason about the uncertainty of those estimates.



1000 pulls,
600 wins
 $Q_t(a_1)=0.6$

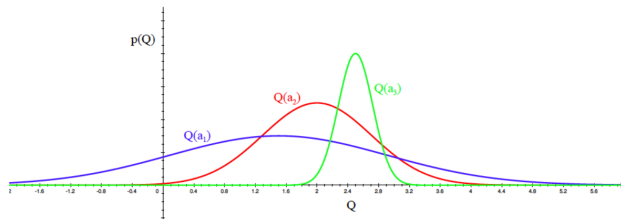


1000 pulls,
400 wins
 $Q_t(a_2)=0.4$



10 pulls,
4 wins
 $Q_t(a_1)=0.4$

Uncertainty Guides Exploration



- The **more uncertain** we are about an action-value
- The **more important** it is to explore that action
- It could turn out to be the best action

Upper Confidence Bounds (UCB)

- Estimate an **upper confidence** $U_t(a)$ for each action value such that with high probability:

$$Q_*(a) \leq Q_t(a) + U_t(a)$$

$Q_t(a)$ is the estimated mean, $U_t(a)$ is the estimated upper confidence

- This upper confidence depends on the times of action a has been selected
 - Small $N_t(a)$, $\Rightarrow$ large $U_t(a)$ (estimated value is uncertain)
 - Large $N_t(a)$, $\Rightarrow$ small $U_t(a)$ (estimated value is accurate)
- Select action maximizing **Upper Confidence Bound (UCB)**

$$a_t = \arg\max_a Q_t(a) + U_t(a)$$

Background: Hoeffding's Inequality

Theorem

Hoeffding's Inequality Let $Z_1, Z_2, \dots, Z_n$ be independent bounded random variables with $Z_i \in [a, b]$ for all i , where $-\infty < a \leq b < \infty$. Then,

$$p\left(\frac{1}{n} \sum_{i=1}^n (Z_i - \mathbb{E}[Z_i]) \geq t\right) \leq \exp\left(-\frac{2nt^2}{(b-a)^2}\right)$$

and

$$p\left(\frac{1}{n} \sum_{i=1}^n (Z_i - \mathbb{E}[Z_i]) \leq -t\right) \leq \exp\left(-\frac{2nt^2}{(b-a)^2}\right)$$

for all $t \geq 0$.

Hoeffdings Inequality

- We will apply Hoeffdings Inequality to the rewards obtained from each action (bandit) a :

$$p(Q_t(a) - U_t(a) \geq Q_*(a)) \leq \exp(-2N_t(a)U_t(a)^2)$$

$$p(Q_t(a) + U_t(a) \leq Q_*(a)) \leq \exp(-2N_t(a)U_t(a)^2)$$

- t : how many times I have played any action
- $N_t(a)$: how many times I have played the action a in t interactions
- $U_t(a)$: made the margin to depend on the amount of interactions t

Calculating Upper Confidence Bounds

- Pick a probability p that the value estimate deviates from its mean
- Now solve for $U_t(a)$

$$e^{-2U_t(a)^2 N_t(a)} = p$$

$$U_t(a) = \sqrt{\frac{-\log p}{2N_t(a)}}$$

- Reduce p as we play more, e.g., $p = t^{-k}$, e.g., $k = 2$
- Ensure we select optimal action as $t \rightarrow \infty$

$$U_t(a) = \sqrt{\frac{\log t}{N_t(a)}}$$

Upper Confidence Bound (UCB)

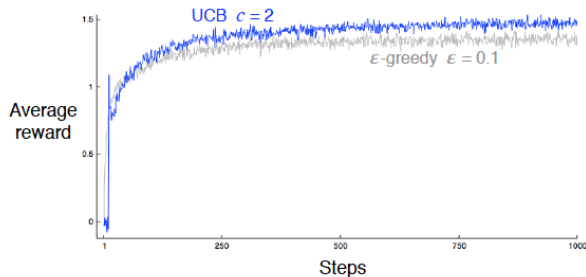
- A clever way of reducing exploration over time
- Estimate an upper bound on the true action values
- Select the action with the largest (estimated) upper bound

$$a_t = \arg\max_a \left[Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right]$$

- c is a hyper-parameter that **trades-off exploration and exploitation**
- CB grows with the total number of actions we have taken t but shrinks with the number of times we have tried this particular action $N_t(a)$.
- **This ensures each action is tried infinitely often but still balances exploration/exploitation.**

Upper Confidence Bound (UCB)

$$a_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right]$$



- 1 Multi-Armed Bandits
- 2 Basic Concepts in MAB
- 3 Action Selection in MAB
- 4 Upper Confidence Bounds
- 5 Gradient Bandits**

Gradient Bandits

- We have m **score** parameters (to be optimized), one for each action, denoted as $\{s_a | a \in A\}$
- $\pi : A \rightarrow [0, 1]$ refers to the function for **action-selection probabilities**, that is defined as follows:

$$\pi(a) = \frac{e^{s_a}}{\sum_{b \in A} e^{s_b}}, \quad \forall a \in A$$

- i.e., $\pi(a)$ is a softmax function of the score parameter.
- Expected reward objective:

$$J(s_{a_1}, s_{a_2}, \dots, s_{a_m}) = \sum_{a \in A} \pi(a) \cdot \mathbb{E}[r | a]$$

Calculate Gradient of $J(\cdot)$

We calculate $\frac{\partial J}{\partial s_a}$ for each $a \in \mathbf{A}$:

$$\begin{aligned}\frac{\partial J}{\partial s_a} &= \frac{\partial \left(\sum_{a \in \mathbf{A}} \pi(a) \cdot \mathbb{E}[r|a] \right)}{\partial s_a} \\ &= \sum_{a \in \mathbf{A}} \mathbb{E}[r|a] \cdot \frac{\partial \pi(a')}{\partial s_a} \\ &= \sum_{a \in \mathbf{A}} \pi(a') \cdot \mathbb{E}[r|a] \cdot \frac{\partial \log \pi(a')}{\partial s_a} \\ &= \mathbb{E}_{a' \sim \pi, r \sim r(a')} \left[r \cdot \frac{\partial \log \pi(a')}{\partial s_a} \right]\end{aligned}$$

Calculate Gradient of $J(\cdot)$

$$\frac{\partial J}{\partial s_a} = \mathbb{E}_{a' \sim \pi, r \sim r(a')} \left[r \cdot \frac{\partial \log \pi(a')}{\partial s_a} \right]$$

- We know from standard softmax-function calculus that:

$$\frac{\partial \log \pi(a')}{\partial s_a} = \frac{\partial \left(\log \frac{e^{s_a}}{\sum_{b \in A} e^{s_b}} \right)}{\partial s_a} = \mathbb{1}_{a=a'} - \pi(a)$$

- Therefore, $\frac{\partial \log \pi(a')}{\partial s_a}$ can be re-written as:

$$\frac{\partial \log \pi(a')}{\partial s_a} = \mathbb{E}_{a' \sim \pi, r \sim r(a')} [r \cdot (\mathbb{1}_{a=a'} - \pi(a))]$$

- **Gradient Bandits:** at each time step t , we approximate the gradient with

Extra Reading Materials

- Chapter 2 (Multi-armed Bandits), Reinforcement Learning: An Introduction, Sutton & Barto.
- Bayesian Bandits and Thompson Sampling.